

Modul Next.JS — Chapter 4 s/d 6

Mata Kuliah: Sistem Informasi Web (SIWEB)

Topik: Routing, Navigation & Database Setup

Tahun Akademik: 2025/2026

Daftar Isi

1. [Chapter 4 — Creating Layouts and Pages](#)
2. [Chapter 5 — Navigating Between Pages](#)
3. [Chapter 6 — Setting Up Your Database](#)

CHAPTER 4

Creating Layouts and Pages

Di chapter ini, kita mulai membangun halaman-halaman pada dashboard app! Kita akan belajar bagaimana Next.JS menggunakan file-system routing untuk membuat route baru, serta membuat layout yang bisa dipakai bersama oleh beberapa halaman.

Yang Akan Kita Pelajari

- Membuat route `dashboard` menggunakan file-system routing
- Memahami peran folder dan file dalam membuat route segment baru
- Membuat nested layout yang bisa dipakai bersama beberapa halaman
- Memahami konsep colocation, partial rendering, dan root layout

Nested Routing

Sebelum kita mulai membuat halaman baru, kita perlu pahami dulu konsep routing di Next.JS. Berbeda dengan framework lain, Next.JS menggunakan **file-system routing** yang artinya struktur folder kalian = struktur URL aplikasi kalian.

Jadi, setiap folder yang dibuat di dalam direktori `/app` akan menjadi sebuah **route segment** yang memetakan ke URL segment yang sesuai. Dan untuk membuat route yang bisa diakses publik, kita perlu membuat file `page.tsx` di dalam folder tersebut.

💡 Kenapa harus pakai `page.tsx`?

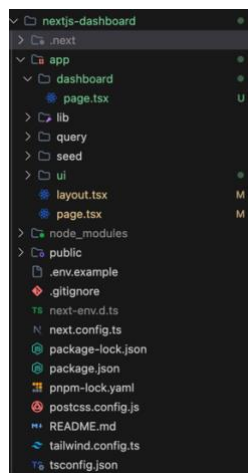
Di Next.JS, hanya file yang bernama `page.tsx` yang akan jadi halaman yang bisa diakses publik.

File-file lain seperti komponen, utility, dan test file bisa kamu letakkan di folder yang sama dan tidak akan jadi route. Keren kan? Ini namanya **colocation**!

Membuat Dashboard Page

Oke sekarang kita mulai praktek! Pertama-tama, buat folder baru bernama `dashboard` di dalam folder `/app`. Selanjutnya, buat file `page.tsx` di dalam folder `dashboard` tersebut.

Struktur folder yang diharapkan:



Isi file `/app/dashboard/page.tsx`:

```
nextjs-dashboard > app > dashboard > page.tsx > Page
1  export default function Page() {
2    return <p>Dashboard Page</p>
3  }
```

Sekarang pastikan development server kalian berjalan (kalau belum ketik perintah di terminal `pnpm dev`), lalu buka browser dan kunjungi `http://localhost:3000/dashboard`. Kalian akan melihat tulisan **"Dashboard Page"**.

Membuat Dashboard Layout

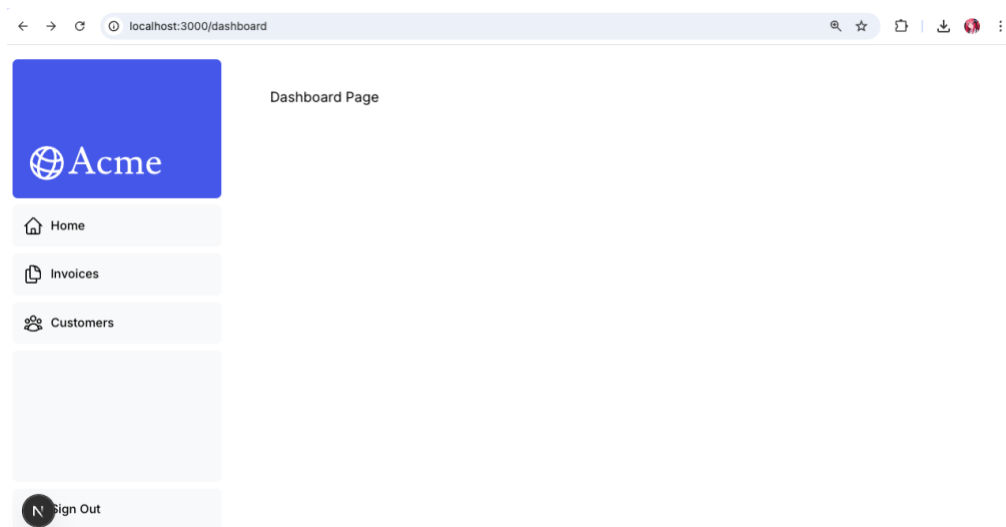
Dashboard umumnya punya navigasi yang dipakai bersama di berbagai halaman. Nah, di Next.JS kita bisa pakai file khusus bernama `layout.tsx` untuk membuat UI yang di-share ke beberapa halaman sekaligus.

Buat file baru bernama `layout.tsx` di dalam folder `/dashboard`, lalu masukkan kode berikut.

File `/app/dashboard/layout.tsx`:

```
nextjs-dashboard > app > dashboard > layout.tsx > Layout
1  import SideNav from '@app/ui/dashboard/sidenav';
2
3  export default function Layout({ children }: { children: React.ReactNode }) {
4    return (
5      <div className="flex h-screen flex-col md:flex-row md:overflow-hidden">
6        <div className="w-full flex-none md:w-64">
7          <SideNav />
8        </div>
9        <div className="grow p-6 md:overflow-y-auto md:p-12">
10         {children}
11       </div>
12     </div>
13   );
14 }
```

Jangan lupa gess dilihat hasilnya di <http://localhost:3000/dashboard> Pastikan hasilnya seperti ini ya



Beberapa hal penting yang perlu kalian perhatikan dari kode di atas:

1. Kita import komponen `<SideNav />` ke dalam layout. Semua komponen yang diimport di sini akan menjadi bagian dari layout.
2. Komponen `<Layout />` menerima prop `children`. Child ini bisa berupa halaman atau layout lain.

Semua halaman di dalam `/dashboard` otomatis akan di-*wrap* oleh layout ini.

⚡ Partial Rendering — Hemat Performa!

Salah satu keuntungan besar pakai layouts di Next.JS adalah ketika kita navigasi antar halaman, hanya komponen halaman yang di-update — layout tidak di-render ulang! Ini disebut **partial rendering** dan membantu menjaga state React di sisi client tetap terjaga.

Root Layout

Di Chapter 3 kemarin, kita sudah menambahkan font Inter ke layout di `/app/layout.tsx`. File inilah yang disebut **root layout** dan wajib ada di setiap aplikasi Next.JS.

File `/app/layout.tsx`:

```

nextjs-dashboard > app > layout.tsx > ...
1  import '@app/ui/global.css';
2  import { inter } from '@app/ui/fonts';
3
4  export default function RootLayout({
5    children,
6  }: {
7    children: React.ReactNode;
8  }) {
9    return (
10     <html lang="en">
11       <body className={` ${inter.className} antialiased`}>{children}</body>
12     </html>
13   );
14 }
15

```

UI yang ditambahkan ke root layout akan di-share ke **semua halaman** dalam aplikasi. Root layout ini juga tempat yang tepat untuk memodifikasi tag ``<html>`` dan ``<body>``, serta menambahkan metadata.

📌 Catatan Penting!

Karena layout baru yang kita buat di `/app/dashboard/layout.tsx` bersifat unik untuk halaman-halaman dashboard saja, kita tidak perlu menambahkan UI apapun ke root layout di atas.

CHAPTER 5

Navigating Between Pages

Di chapter sebelumnya, kita sudah membuat layout dan halaman-halaman dashboard. Sekarang saatnya menambahkan link agar pengguna bisa berpindah antar halaman dengan mulus tanpa full page refresh!

Yang Akan Kita Pelajari

- Cara menggunakan komponen `next/link`
- Cara menampilkan active link dengan hook `usePathname()`
- Cara kerja navigasi di Next.JS

Kenapa Kita Perlu Mengoptimalkan Navigasi?

Untuk menghubungkan antar halaman, secara tradisional kita menggunakan elemen `` HTML biasa. Kalau kalian perhatikan sidebar links yang ada di project ini, sekarang masih pakai `` dan cobalah navigasi antara halaman home, invoices, dan customers.

Kalian pasti sadar kan? **Ada full page refresh setiap kali navigasi!** Ini tidak efisien dan terasa lambat. Nah, di sinilah komponen `<Link>` dari Next.JS hadir sebagai solusi!

Komponen `<Link>`

Next.JS menyediakan komponen `<Link />` yang memungkinkan kita melakukan client-side navigation dengan JavaScript — tidak ada full page reload lagi!

Buka file `/app/ui/dashboard/nav-links.tsx`, kemudian import komponen `Link` dari `next/link` dan ganti semua tag `` dengan ``<Link>`` seperti di bawah ini.

File `/app/ui/dashboard/nav-links.tsx`:

```

1 import {
2   UserGroupIcon,
3   HomeIcon,
4   DocumentDuplicateIcon,
5 } from '@heroicons/react/24/outline';
6 import Link from 'next/link'; // ← Tambahkan import ini!
7
8 // Map of links to display in the side navigation.
9 // Depending on the size of the application, this would be stored in a database.
10 const links = [
11   { name: 'Home', href: '/dashboard', icon: HomeIcon },
12   {
13     name: 'Invoices',
14     href: '/dashboard/invoices',
15     icon: DocumentDuplicateIcon,
16   },
17   { name: 'Customers', href: '/dashboard/customers', icon: UserGroupIcon },
18 ];
19
20 export default function NavLinks() {
21   return (
22     <div>
23       {links.map((link) => {
24         const LinkIcon = link.icon;
25         return (
26           <Link /* ← Ganti <a> dengan <Link> */
27             key={link.name}
28             href={link.href}
29             className="flex h-[48px] grow items-center justify-center gap-2 rounded-md bg-gray-50 p-3
30               text-sm font-medium hover:bg-sky-100 hover:text-blue-600 md:flex-none md:justify-start md:p-2 md:px-3"
31             >
32               <LinkIcon className="w-6" />
33               <p className="hidden md:block">{link.name}</p>
34             </Link> /* ← Ganti </a> dengan </Link> */
35           );
36         });
37     </div>
38   );
39 }
40

```

⚡ Automatic Code-Splitting & Prefetching

Next.JS secara otomatis **memecah kode** (code splitting) berdasarkan route. Artinya setiap halaman jadi terisolasi — jika satu halaman error, halaman lain tetap berjalan! Selain itu, setiap kali komponen `<Link>` terlihat di viewport browser, Next.JS otomatis **prefetch** kode halaman tujuan di background. Hasilnya: transisi halaman hampir instan! 🚀

Menampilkan Active Link

Pola UI yang umum adalah menampilkan link aktif untuk menunjukkan kepada pengguna halaman mana yang sedang mereka buka. Untuk ini, kita butuh tahu path URL saat ini. Next.JS menyediakan hook bernama `usePathname()` yang bisa kita gunakan!

Mengubah Menjadi Client Component

Karena `usePathname()` adalah sebuah React hook, kita perlu mengubah `nav-links.tsx` menjadi **Client Component**. Caranya, tambahkan directive 'use client' di baris paling atas file.

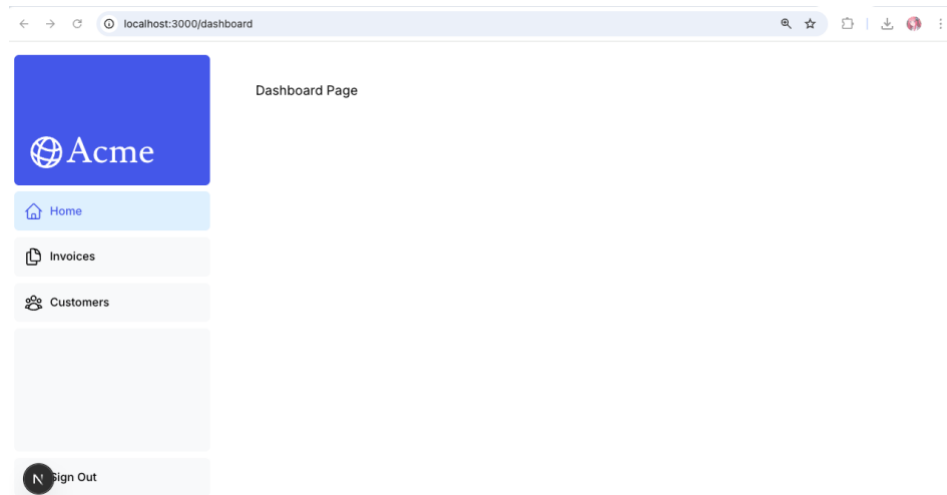
⚠ Server vs Client Component?

Secara default, semua komponen di Next.JS adalah **Server Component**. React Hook seperti `usePathname()` hanya bisa dipakai di **Client Component**. Untuk mengubahnya, tambahkan 'use client' di baris pertama file.

File `/app/ui/dashboard/nav-links.tsx` (Final):

```
nextjs-dashboard > app > ui > dashboard > nav-links.tsx > ...
1  'use client'; // - Tambahkan di baris pertama!
2
3  import {
4    UserGroupIcon,
5    HomeIcon,
6    DocumentDuplicateIcon,
7  } from '@heroicons/react/24/outline';
8  import Link from 'next/link';
9  import { usePathname } from 'next/navigation'; // - Import hook ini
10 import clsx from 'clsx'; // - Kita pakai clsx dari Chapter 2!
11
12 const links = [
13   { name: 'Home', href: '/dashboard', icon: HomeIcon },
14   {
15     name: 'Invoices',
16     href: '/dashboard/invoices',
17     icon: DocumentDuplicateIcon,
18   },
19   { name: 'Customers', href: '/dashboard/customers', icon: UserGroupIcon },
20 ];
21
22 export default function NavLinks() {
23
24   const pathname = usePathname(); // - Dapatkan path saat ini
25   return (
26     <>
27       {links.map((link) => {
28         const LinkIcon = link.icon;
29         return (
30           <Link
31             key={link.name}
32             href={link.href}
33             className={clsx(
34               'flex h-[48px] grow items-center justify-center gap-2 rounded-md bg-gray-50 p-3 text-sm font-medium hover:bg-sky-100 hover:text-blue-600 md:flex-none md:justify-start md:p-2 md:px-3',
35               'bg-sky-100 text-blue-600': pathname === link.href,
36             )}
37           >
38             <LinkIcon className="w-6" />
39             <p className="hidden md:block">{link.name}</p>
40           </Link>
41         );
42       })}
43     </>
44   );
45 }
46
47 }
```


Simpan perubahan dan cek di localhost kalian. Sekarang link yang aktif akan ditampilkan dengan warna biru! 🌸



📝 INTERMEZZO — Ingat clsx dari Chapter 2?

Di Chapter 2 kita sudah berkenalan dengan library `clsx` untuk mengubah class secara kondisional. Di sini kita manfaatkan lagi! Pola `{ 'nama-class': kondisi }` berarti class tersebut hanya ditambahkan jika kondisinya bernilai `true`. Jadi saat `pathname === link.href` nilainya `true`, class `bg-sky-100 text-blue-600` otomatis ditambahkan.

CHAPTER 6

Setting Up Your Database

Sebelum melanjutkan pengembangan dashboard, kita butuh data! Di chapter ini, kita akan setup database PostgreSQL melalui integrasi Vercel, deploy project ke Vercel, dan mengisi database dengan data awal.

Yang Akan Kita Pelajari

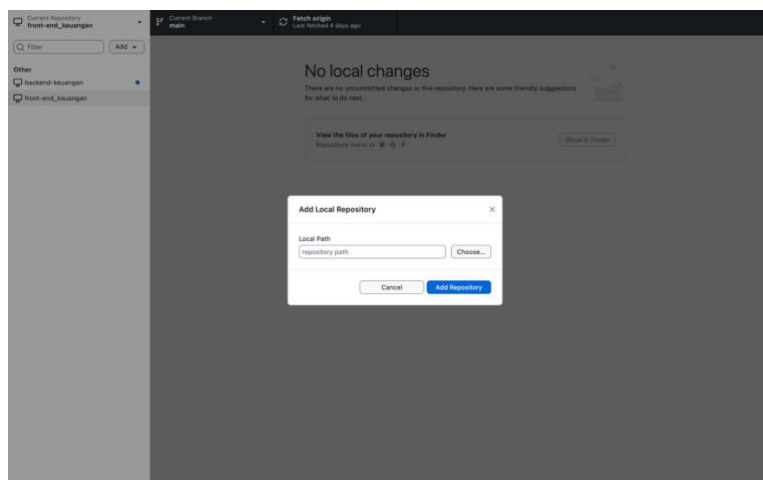
- Push project ke GitHub
- Setup akun Vercel dan hubungkan ke GitHub repo untuk preview & deployment otomatis
- Membuat dan menghubungkan project ke database Postgres
- Mengisi database dengan data awal (seed)

Langkah 1: Push Project ke GitHub

Pertama, kita perlu push repository project kita ke GitHub. Ini penting karena Vercel akan terhubung ke GitHub repo kalian untuk proses deployment.

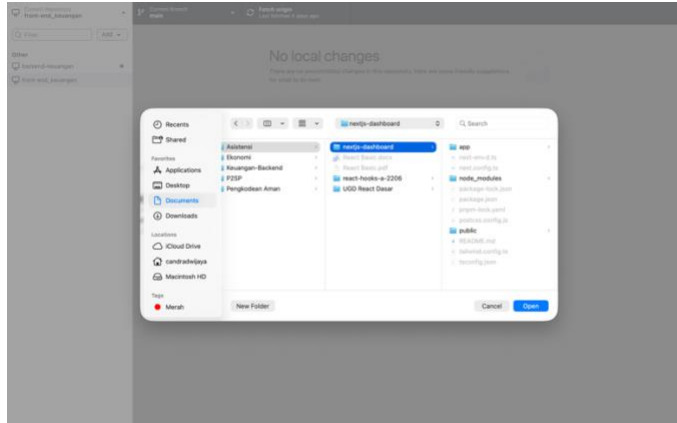
1. Buat repository baru di GitHub

Buka github desktop, klik tombol "add" dan pilih "Add Existing Repository" dan klik "Add Repository".



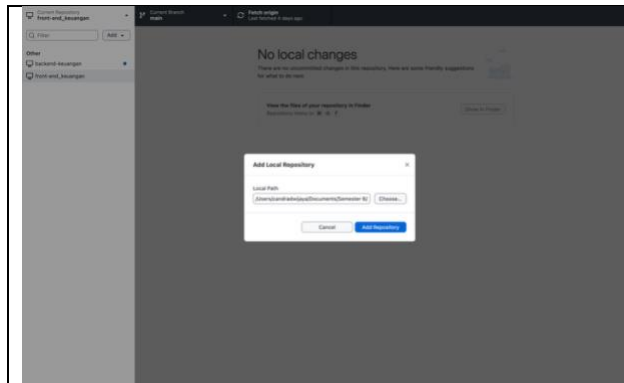
2. Pilih project

Selanjutnya kalian tinggal pilih folder project yang sudah kalian buat, setelah itu klik ok.

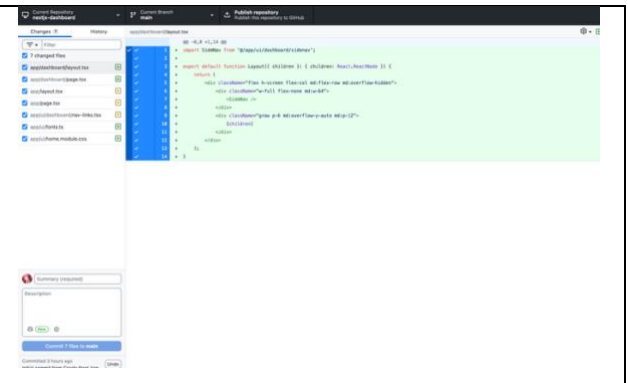


3. Tambah Repository

Setelah kalian klik ok, maka local path otomatis terisi dengan lokasi project kalian, di tahap ini kalian tinggal klik "Add Repository".



"Add Repository".



Tampilan setelah "Add Repository"

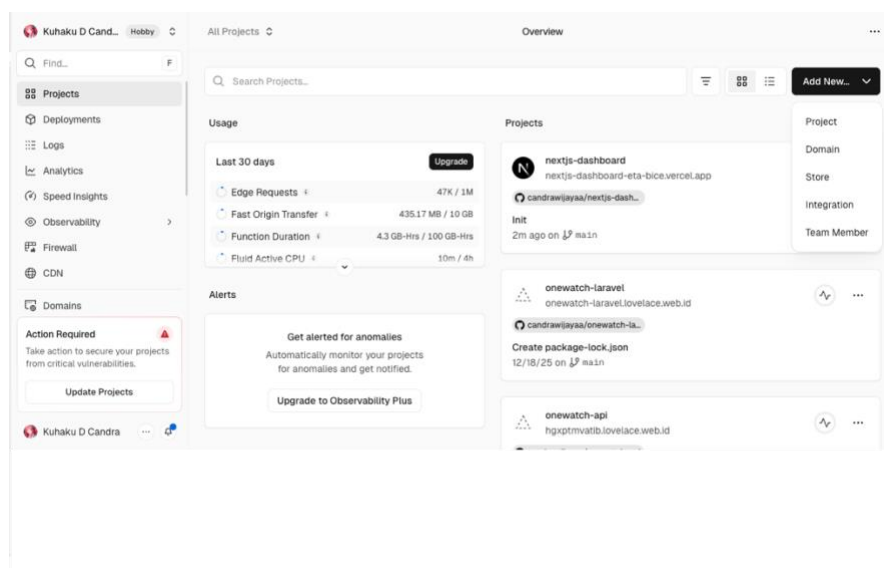
Setelah itu, kalian tinggal masukkan commit dan publish repository, jangan lupa private repositorynya jangan dicentang, supaya bisa diakses public

Langkah 2: Setup Vercel & Deploy Project

Vercel adalah platform cloud yang dibuat oleh tim yang sama dengan Next.JS, jadi integrasinya sangat mulus! Kalian bisa deploy project Next.JS dengan sangat mudah di sini.

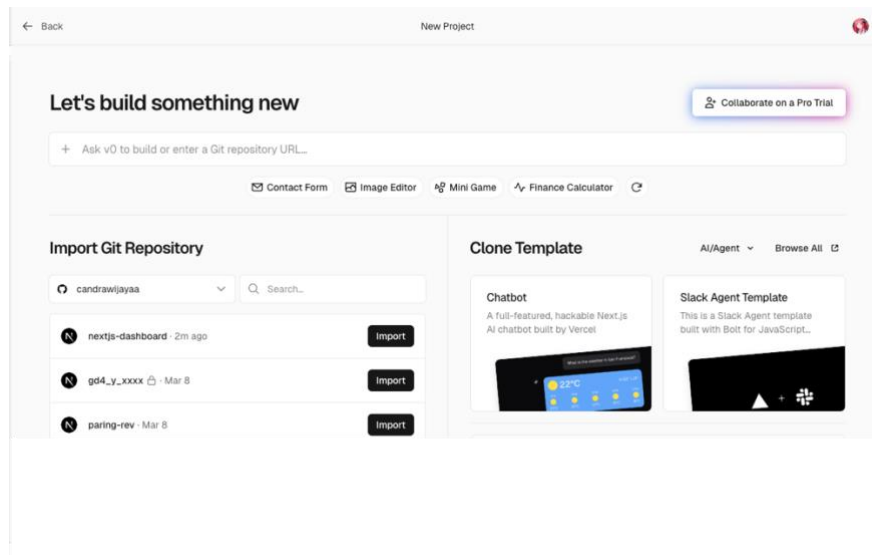
1. Masuk ke akun Vercel

Kunjungi vercel.com dan masuk ke akun kalian. Setelah kalian masuk ke vercel tambah Project yang tadi telah kalian push ke github.



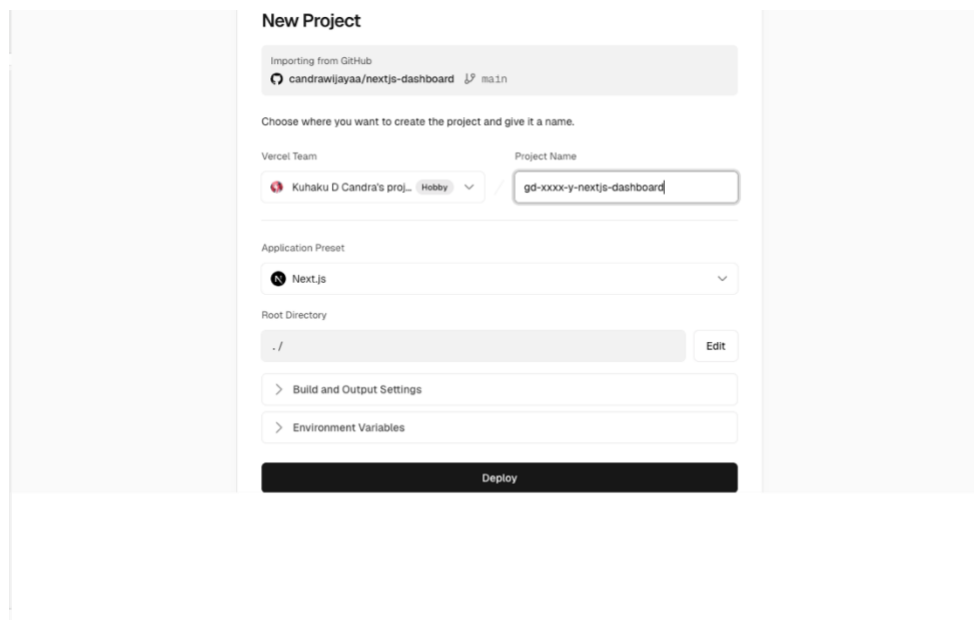
2. Import GitHub Repository

Setelah login, kalian akan diarahkan ke dashboard. Klik **"Import Project"**, lalu pilih repository [nextjs-dashboard](#) yang baru saja kalian push tadi.



3. Deploy!

Beri nama project kalian `gd-xxxx-y-nextjs-dashboard` (xxxx adalah 4 digit npm terakhir dan y adalah kelas), lalu klik **"Deploy"**. Tunggu beberapa saat hingga proses deployment selesai.



Auto-Deploy dengan GitHub!

Dengan menghubungkan GitHub repository ke Vercel, setiap kali kalian git push ke branch **main**, Vercel akan otomatis re-deploy aplikasi kalian tanpa konfigurasi tambahan. Selain itu,

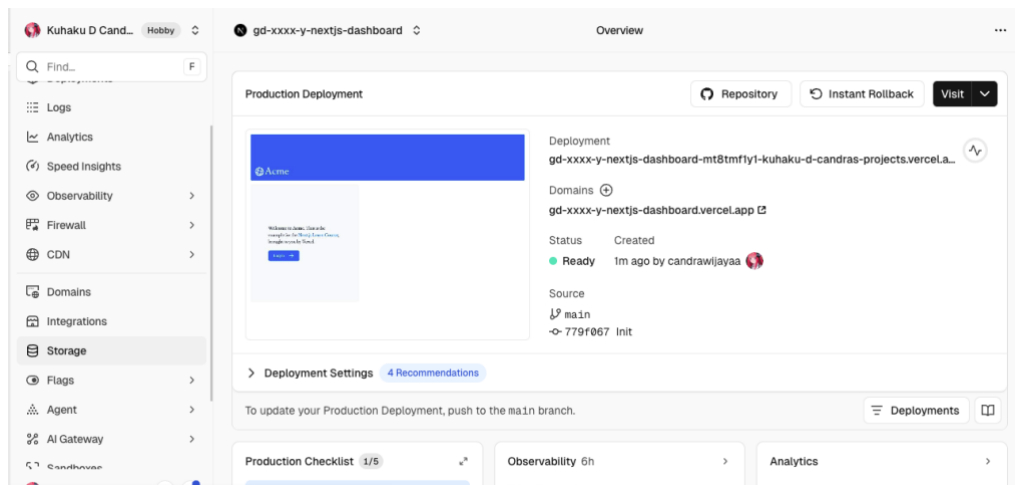
setiap Pull Request juga akan mendapat preview URL otomatis — sangat berguna untuk review kode!

Langkah 3: Membuat Database Postgres

Setelah deploy berhasil, saatnya membuat database! Di Vercel, kita bisa membuat database PostgreSQL dengan sangat mudah.

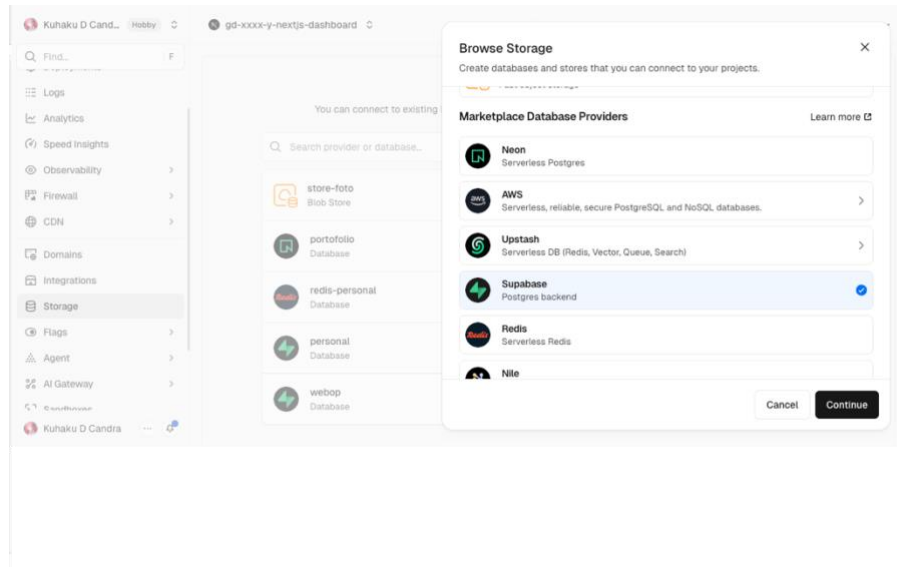
1. Buka tab Storage di Vercel Dashboard

Klik **"Continue to Dashboard"**, lalu pilih tab **"Storage"** dari project dashboard kalian.



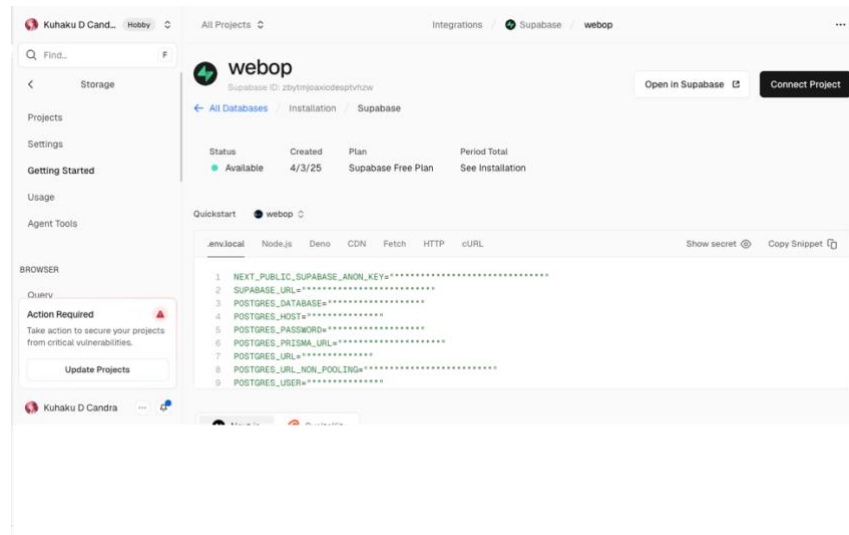
2. Buat Database Baru

Klik **"Create Database"**. Pilih provider yang tersedia (Neon atau Supabase). Pilih region **Washington D.C (iad1)** untuk latency yang lebih rendah. Klik **"Create"**.



3. Copy Environment Variables

Navigasi ke tab **".env.local"**, klik **"Show secret"** lalu **"Copy Snippet"**. Pastikan kalian sudah klik Show secret sebelum copy!



4. Paste ke file .env lokal

Di code editor, rename file `.env.example` menjadi `.env`, lalu paste konten yang sudah di-copy dari Vercel tadi ke dalamnya.

```
1 NEXT_PUBLIC_SUPABASE_ANON_
2 NEXT_PUBLIC_SUPABASE_PUBLI
3 NEXT_PUBLIC_SUPABASE_URL="
4 POSTGRES_DATABASE="postgre
5 POSTGRES_HOST="db.tlgcrvwp
6 POSTGRES_PASSWORD="sgzYCK7
7 POSTGRES_PRISMA_URL="postg
8 POSTGRES_URL="postgres://p
9 POSTGRES_URL_NON_POOLING="
10 POSTGRES_USER="postgres"
11 SUPABASE_ANON_KEY="eyJhbGc
12 SUPABASE_JWT_SECRET="WajZj
13 SUPABASE_PUBLISHABLE_KEY="
14 SUPABASE_SECRET_KEY="sb_se
15 SUPABASE_SERVICE_ROLE_KEY=
16 SUPABASE_URL="https://tlgc
```

📝 INTERMEZZO — Jangan Sampai Bocor! Lindungi File .env

File `.env` berisi credential database kalian yang sangat sensitif. Pastikan file ini **TIDAK** ikut di-push ke GitHub! Buka file `.gitignore` dan pastikan `.env` sudah ada di dalam daftar file yang di-ignore.

```
# environment variables

.env

.env.local

.env.development.local

.env.test.local

.env.production.local
```

Langkah 4: Seed Database

Database sudah dibuat, sekarang kita perlu mengisinya dengan data awal! Project kita sudah dilengkapi dengan API route yang bisa menjalankan seed script untuk mengisi database.

1. Perbaiki Seed Route

Masuk ke file `/app/seed/route.ts`


```

nextjs-dashboard > app > seed > TS route.ts > seedInvoices
1  import bcrypt from 'bcrypt';
2  import postgres from 'postgres';
3  import { invoices, customers, revenue, users } from '../lib/placeholder-data';
4
5  const sql = postgres(process.env.POSTGRES_URL!, { ssl: 'require' });
6
7  async function seedUsers(sql: any) {
8    await sql`CREATE EXTENSION IF NOT EXISTS "uuid-oss";
9    await sql`
10     CREATE TABLE IF NOT EXISTS users (
11       id UUID DEFAULT uuid_generate_v4() PRIMARY KEY,
12       name VARCHAR(255) NOT NULL,
13       email TEXT NOT NULL UNIQUE,
14       password TEXT NOT NULL
15     );
16   `;
17
18   const insertedUsers = await Promise.all(
19     users.map(async (user) => {
20       const hashedPassword = await bcrypt.hash(user.password, 10);
21       return sql`
22         INSERT INTO users (id, name, email, password)
23         VALUES (${user.id}, ${user.name}, ${user.email}, ${hashedPassword})
24         ON CONFLICT (id) DO NOTHING;
25       `;
26     }),
27   );
28
29   return insertedUsers;
30 }
31
32 async function seedInvoices(sql: any) {
33   await sql`CREATE EXTENSION IF NOT EXISTS "uuid-oss";
34
35   await sql`
36     CREATE TABLE IF NOT EXISTS invoices (
37       id UUID DEFAULT uuid_generate_v4() PRIMARY KEY,
38       customer_id UUID NOT NULL,
39       amount INT NOT NULL,
40       status VARCHAR(255) NOT NULL,
41       date DATE NOT NULL
42     );
43   `;

```

Pada line 7, 32, 58, 83, didalam function () tambahkan sql:any, seperti pada gambar disamping

Dan tambahkan const seperti pada line 106 - 111

```

103
104 export async function GET() {
105   try {
106     const result = await sql.begin(async (sql) => {
107       await seedUsers(sql);
108       await seedCustomers(sql);
109       await seedInvoices(sql);
110       await seedRevenue(sql);
111     });
112
113     return Response.json({ message: 'Database seeded successfully' });
114   } catch (error) {
115     return Response.json({ error }, { status: 500 });
116   }
117 }

```

2. Akses endpoint seed

Buka browser dan navigasi ke `http://localhost:3000/seed`. Tunggu hingga selesai — kalian akan melihat pesan **"Database seeded successfully"**.



Langkah 5: Executing Queries

Terakhir, mari verifikasi bahwa database sudah berfungsi dengan benar dengan menjalankan query. Kita akan memanfaatkan Route Handler yang sudah ada di `app/query/route.ts`.

Di dalam file tersebut, terdapat fungsi `listInvoices()` yang berisi query SQL berikut. Uncomment kode di file tersebut terlebih dahulu, kemudian hapus blok `Response.json()`.

```
nextjs-dashboard > app > query > TS route.ts > ...
1  import postgres from 'postgres';
2
3  const sql = postgres(process.env.POSTGRES_URL!, { ssl: 'require' });
4
5  async function listInvoices() {
6    const data = await sql`
7      SELECT invoices.amount, customers.name
8      FROM invoices
9      JOIN customers ON invoices.customer_id = customers.id
10     WHERE invoices.amount = 666;
11   `;
12
13   return data;
14 }
15
16 export async function GET() {
17   try {
18     return Response.json(await listInvoices());
19   } catch (error) {
20     return Response.json({ error }, { status: 500 });
21   }
22 }
```

Setelah itu, navigasi ke `http://localhost:3000/query` di browser kalian. Jika semuanya berhasil, kalian akan melihat data invoice amount dan name yang dikembalikan oleh query, seperti berikut:



```
localhost:3000/query
pretty-print [x]
[{"amount":666,"name":"Evil Rabbit"}]
```